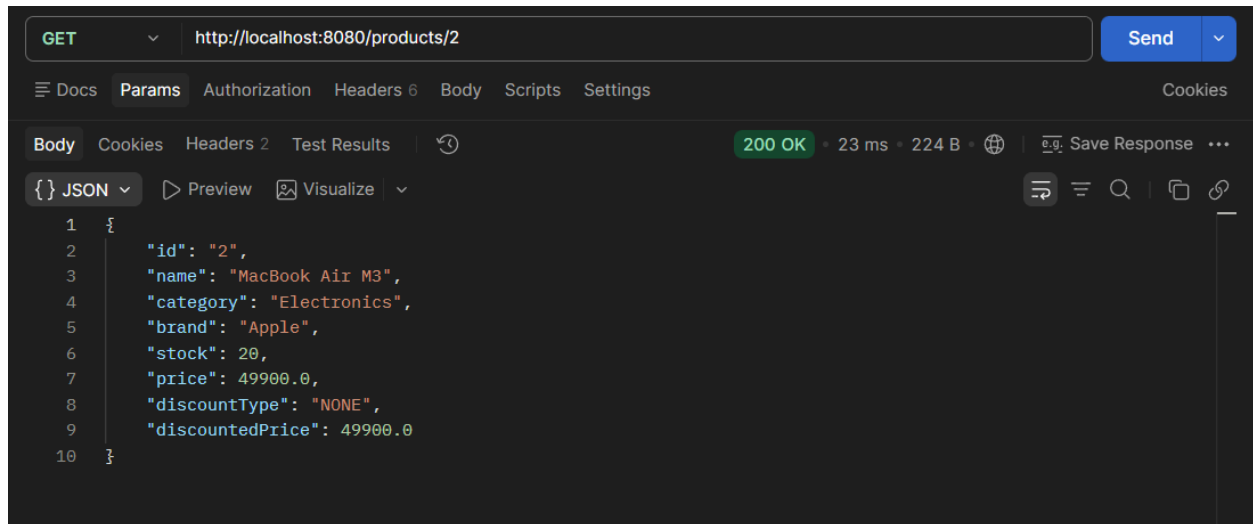
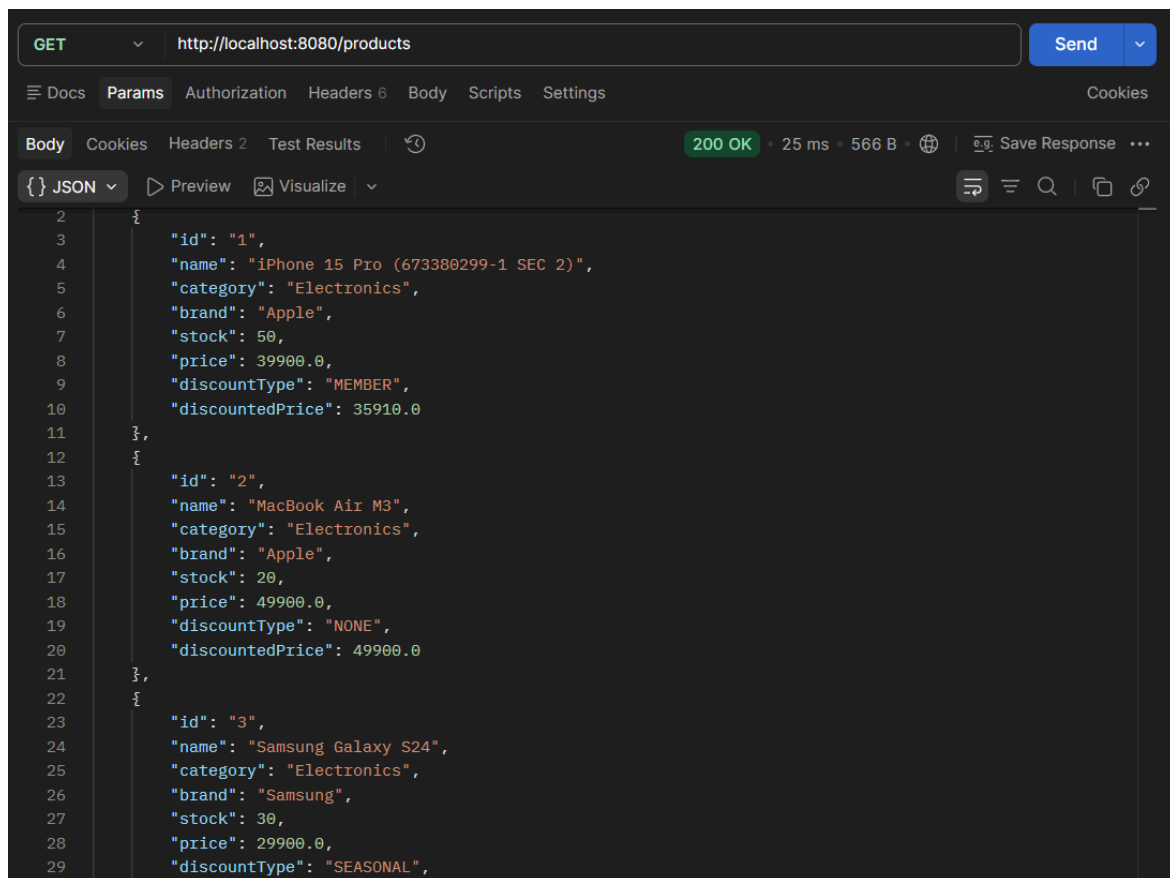


รายงาน Lab 10

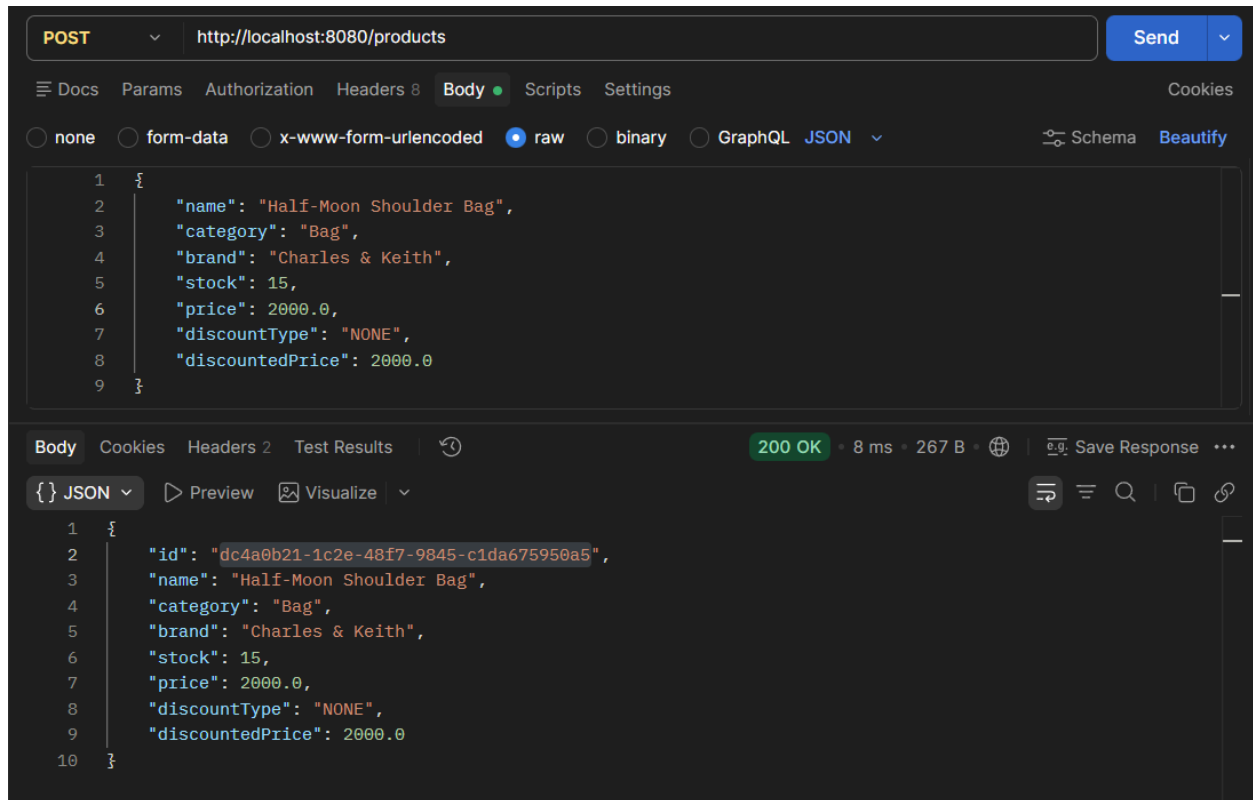
GET /products/{id}



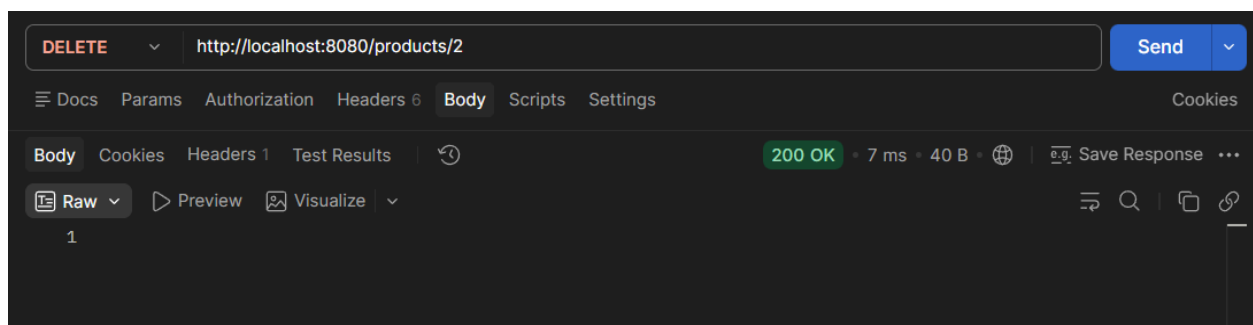
GET /products



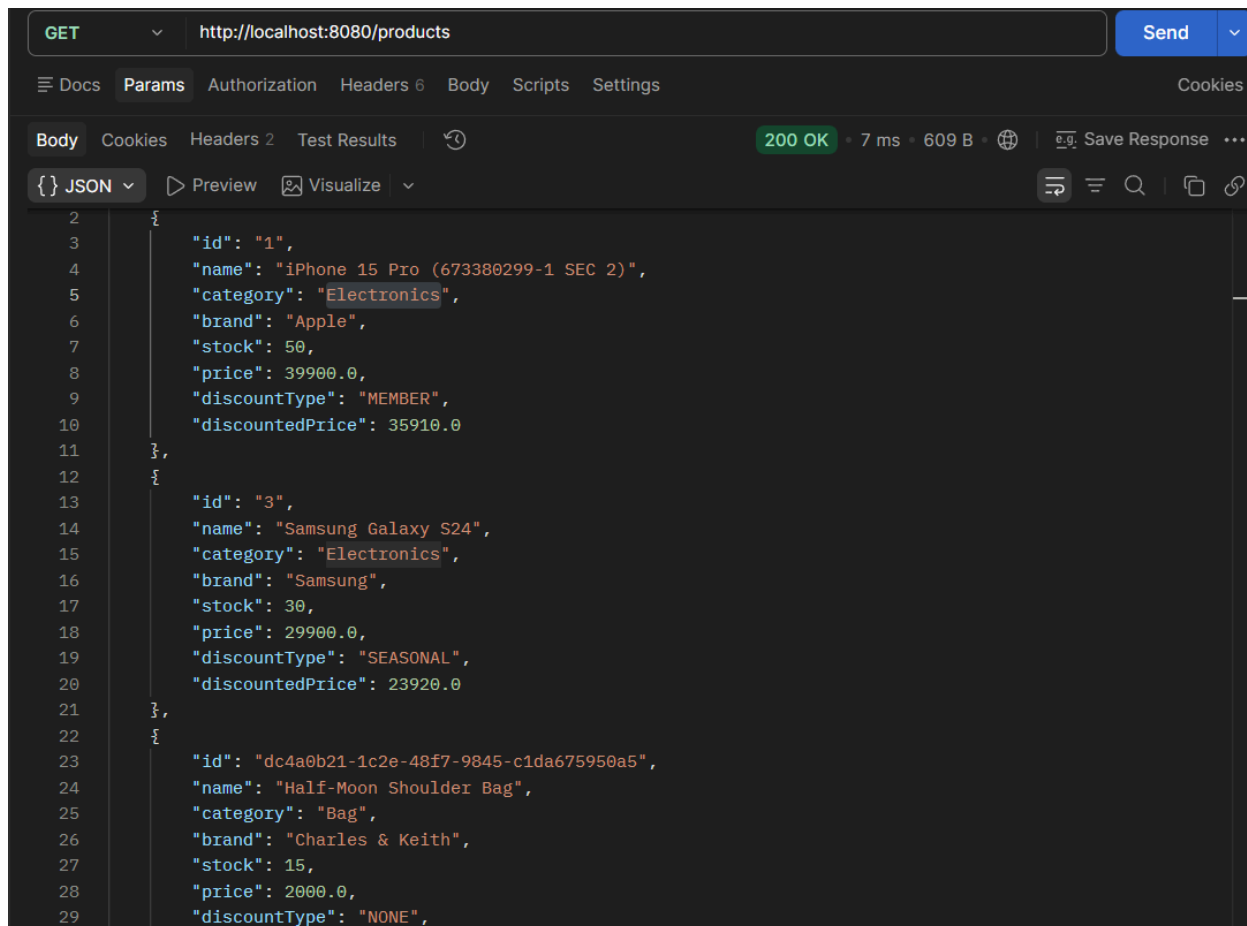
POST /products



DELETE /products/{id}



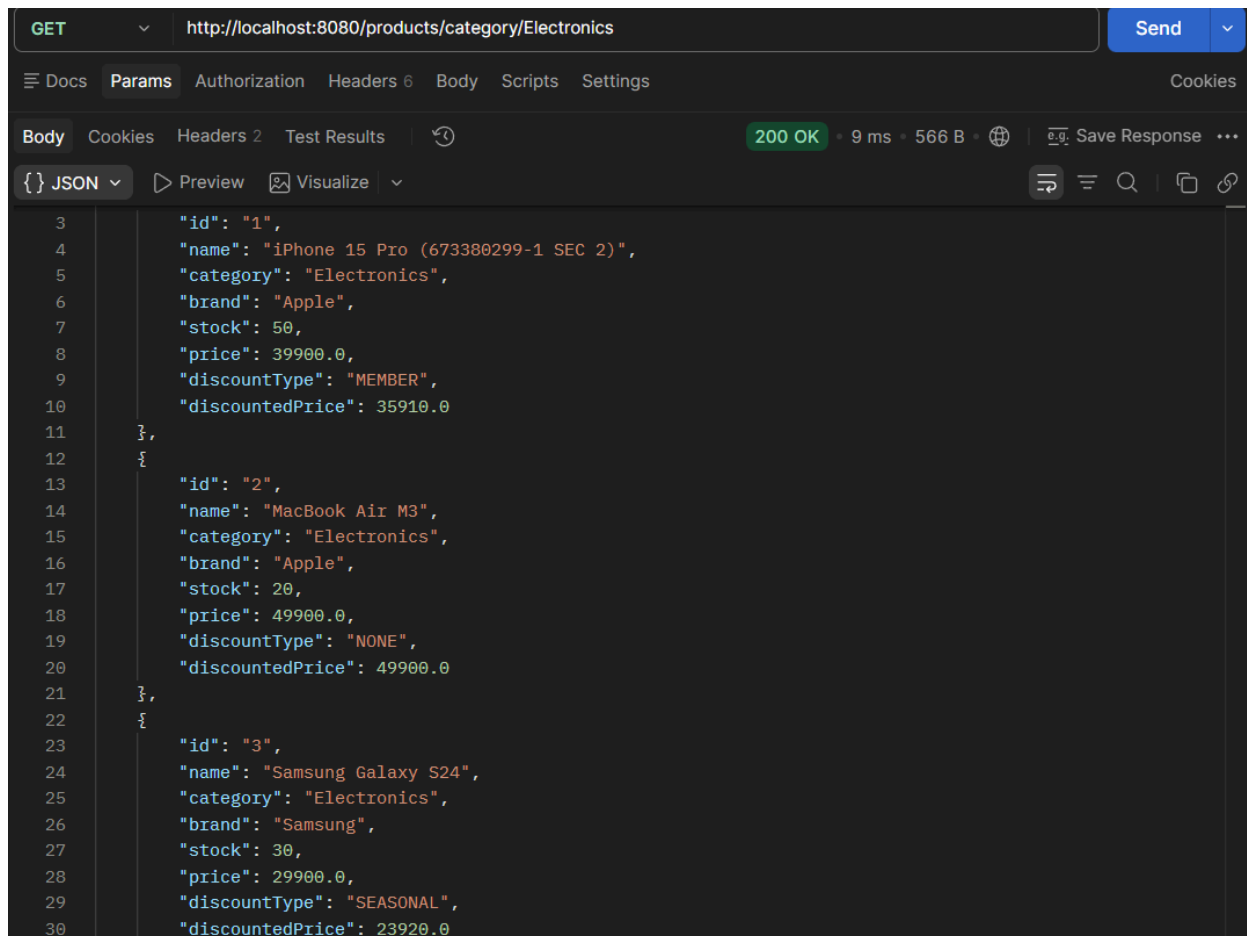
ตรวจสอบรายการสินค้า อีกครั้ง



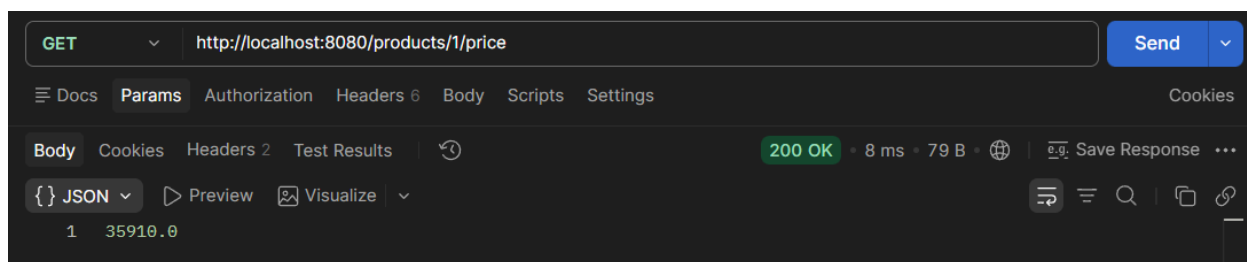
```
GET http://localhost:8080/products Send
Docs Params Authorization Headers 6 Body Scripts Settings Cookies
Body Cookies Headers 2 Test Results 200 OK • 7 ms • 609 B • Save Response ...
JSON Preview Visualize
{
  "id": "1",
  "name": "iPhone 15 Pro (673380299-1 SEC 2)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900.0,
  "discountType": "MEMBER",
  "discountedPrice": 35910.0
},
{
  "id": "3",
  "name": "Samsung Galaxy S24",
  "category": "Electronics",
  "brand": "Samsung",
  "stock": 30,
  "price": 29900.0,
  "discountType": "SEASONAL",
  "discountedPrice": 23920.0
},
{
  "id": "dc4a0b21-1c2e-48f7-9845-c1da675950a5",
  "name": "Half-Moon Shoulder Bag",
  "category": "Bag",
  "brand": "Charles & Keith",
  "stock": 15,
  "price": 2000.0,
  "discountType": "NONE",
}
```

จากภาพจะเห็นว่าสินค้าซึ่งมีไอดีเท่ากับ 2 ถูกลบออกแล้ว

GET /products/category/{cat}



GET /products/{id}/price



1. Reactive Programming vs Blocking — ต่างกันอย่างไร

ตอบ

Reactive Programming คือ การออกแบบให้โปรแกรมสามารถทำงานที่ต้องรอ I/O โดยไม่จำเป็นต้องบล็อก Thread ไว้รอผลลัพธ์

Blocking Programming หรือ การทำงานแบบบล็อก คือ เมื่อโปรแกรมสั่งงานที่ต้องรอ โปรแกรมจะหยุดรอจนกว่างานนั้นจะเสร็จ จากนั้นจึงทำงานต่อ

กล่าวโดยสรุปคือ Blocking ทำงานแบบรอได้ผลลัพธ์แล้วจึงทำงานต่อ ขณะที่ Reactive มีการจัด pipeline ประมวลผลเมื่อข้อมูลพร้อม ทำให้ระบบสามารถใช้ Thread ได้มีประสิทธิภาพมากขึ้น

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ตอบ Mono และ Flux เป็น Reactive Type จาก Project Reactor ซึ่งเป็นพื้นฐานสำคัญของ Spring WebFlux

โดย

Mono<T> ใช้แทนข้อมูล 0 หรือ 1 ค่า เหมาะกับการค้นหาด้วย id เพราะ id ควรแทนข้อมูลรายการเดียว

Flux<T> ใช้แทนข้อมูล 0 ถึงหลายค่า เหมาะกับการค้นหารายการทั้งหมด เพราะสามารถมีข้อมูลได้หลายรายการ

สามารถสรุปการใช้ในกรณีต่างๆ โดยยกตัวอย่างจากแลปได้ดังนี้

กรณี	Reactive Type	เหตุผล
ค้นหา Product ตาม ID	Mono<Product>	ได้ 0–1 Product
ค้นหา Product ทั้งหมด	Flux<Product>	ได้หลาย Product
บันทึก Product	Mono<Product>	คืน Product ที่บันทึก 1 รายการ
ลบ Product	Mono<Void>	ทำงานเสร็จแต่ไม่มีข้อมูลคืน
ค้นหาตาม Category	Flux<Product>	Category หนึ่งมีหลาย Product
ราคาหลังส่วนลด	Mono<Double>	ได้ราคาหนึ่งค่า

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

ตอบ

WebClient คือ HTTP Client ของ Spring WebFlux ใช้สำหรับให้ Application เรียก REST API
 อื่นแบบ Reactive

ตัวอย่าง method chain เขียนได้ดังภาพ

```
public Mono<Product> getProductById(String id) {
    return client.get()
        .uri("/products/{id}", id)
        .retrieve()
        .bodyToMono(Product.class);
}
```

```
public Flux<Product> getAllProducts() {
    return client.get()
        .uri("/products")
        .retrieve()
        .bodyToFlux(Product.class);
}
```

โดย

- .get()** กำหนดว่า Request ที่กำลังสร้างเป็น HTTP Method GET
 หรือหากต้องการส่งข้อมูลจะใช้ **.post()**
- .uri()** กำหนด URL หรือ Path ที่ต้องการเรียก
- .retrieve()** ใช้สำหรับ ส่ง Request และเตรียมรับ Response
- .bodyToMono()** ใช้เมื่อ Response Body มีข้อมูล 0 หรือ 1 ค่า
 ส่วน **.bodyToFlux()** ใช้เมื่อ Response Body มีข้อมูล หลายรายการ

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย
ตอบ

Repository ไม่ควรรับผิดชอบเรื่อง HTTP หรือ Business Logic เพราะหน้าที่หลักคือ จัดการข้อมูล

```
// — 1. หา Product 1 รายการ —————
// ค้นหา Product จาก id
// Mono เพราะผลลัพธ์มีได้ 0 หรือ 1 Product
public Mono<Product> findById(String id) {
    Product product = store.get(id);

    if (product == null) {
        return Mono.empty(); // ถ้าไม่พบ ให้คืน Mono ว่าง
    }

    return Mono.just(product); // ถ้าพบ ให้ห่อ Product ด้วย Mono
}

// — 2. หา Product ทั้งหมด —————
// Flux เพราะสามารถมี Product หลายรายการ
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}
```

```
// — 3. บันทึก Product —————
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);

    return Mono.just(product); // คืน Product ที่บันทึก
}

// — 4. ลบ Product —————
public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty(); // ไม่มีข้อมูลที่ต้องคืนหลังจากลบ
}

// — 5. กรองตาม category —————
public Flux<Product> findByCategory(String category) {
    return findAll() // กรองเฉพาะ Product ที่ Category ตรงกัน
        .filter(p -> p.getCategory().equalsIgnoreCase(category));
}
```

Service ทำหน้าที่ Business Logic Layer และเป็นตัวกลางระหว่าง Controller กับ Repository

```
// — 1. ดึง Product 1 รายการ —————
public Mono<Product> getById(String id) {
    return repository.findById(id) // ค้นหาจาก id
        // ถ้า Repository ไม่พบข้อมูล
        // ให้เปลี่ยนจาก Mono.empty() เป็น Error
        .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));
}

// — 2. ดึง Product ทั้งหมด —————
public Flux<Product> getAll() {
    return repository.findAll();
}

// — 3. บันทึก Product —————
public Mono<Product> save(Product product) {
    // ถ้า Product ยังไม่มี id ให้สร้าง id ใหม่
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
    }
    // ส่ง Product ให้ Repository บันทึก
    return repository.save(product);
}
```

```
// — 4. ลบ Product —————
public Mono<Void> delete(String id) {
    return repository.deleteById(id); // ลบ Product ตาม id
}

// — 5. กรองตาม category —————
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category); // ค้นหา Product ตาม Category
}

// — 6. คำนวณราคาหลังส่วนลด —————
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id) // getById() ได้ Mono<Product>
        .map(p -> p.getDiscountedPrice()); // แปลง Product เป็นราคาหลังส่วนลด
}
```

Controller มีหน้าที่รับ HTTP Request แล้วเรียก Service ไม่ควรใส่ Business Logic ไว้ใน Controller

```
// GET /products/{id}
// ค้นหา Product 1 รายการ
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}

// GET /products
// ค้นหา Product ทั้งหมด
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

// POST /products
// รับ Product จาก Request Body
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

// DELETE /products/{id}
// ลบ Product
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

// GET /products/category/{category}
// ค้นหา Product ตาม Category
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

// GET /products/{id}/price
// คำนวณราคาหลังส่วนลด
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
```